

lab8

Task 1

When dealing with a massive candidate set (e.g., 500 million items), the standard greedy algorithm becomes painfully slow because it examines about 500 million items in each iteration to choose just a single item.

To significantly decrease computation time at the cost of slight subset quality deterioration, you can implement the following ideas:

- **Stochastic/Sampled Evaluation:** Instead of calculating the contribution of every single remaining item in each iteration, randomly sample a smaller subset (e.g., 1,000 items) from the available pool and only evaluate those. You select the best item from that small sample. This causes a "slight deterioration" because you might miss the absolute optimal item in that step, but it drastically cuts down the search space.
- **Lazy Greedy Evaluation:** You can leverage the mathematical feature that the "contribution of each item never increases" as the subset grows. If an item's previously calculated contribution from an earlier step is already smaller than the highest contribution found in the current step, you do not need to recalculate it. While this doesn't deteriorate quality, combining it with sampling makes the algorithm exceptionally fast.

Task 2

The Background & Core Pain Point

The core issue in this task is that when the candidate set is enormous (e.g., 500 million items), randomly selecting a new item to add to the current subset during the local improvement phase is highly ineffective. You need to propose a smarter, more deliberate selection strategy.

In the local improvement algorithm defined in your slides, each iteration consists of two steps:

1. Randomly add one of the remaining items to the selected item set.
2. Remove the worst item with the smallest contribution.

Why does "pure random" fail here? In a massive search space, the density of high-quality solutions is extremely low. If you blindly draw from 500 million remaining candidates, the probability of picking an item that actually improves the objective function (like the Hypervolume indicator) is incredibly slim. As emphasized in the slides, random selection is not efficient because many items (like Item 5 or Item 7 in the visual examples) offer very poor incremental contributions. This causes the algorithm to waste the vast majority of its iterations doing useless work—an item is added, evaluated, and immediately removed because it is the worst item.

Efficient Selection Strategies (Alternatives to Pure Random)

To improve the hit rate of your local search without significantly increasing the computational burden, consider implementing the following optimization strategies:

1. Maintain a "Restricted Elite Pool"

- In the initial stages, the greedy algorithm evaluates each item separately to find the single best item. You can leverage this initial computation by caching the top M items (e.g., the top 10,000) with the highest initial values into a separate array or priority queue.
- During the local improvement phase, **instead of randomly sampling from the entire pool of 500 million items, sample strictly from this elite pool**. This relies on the heuristic assumption that "a globally good item is highly likely to be a good item locally," instantly shrinking your search space by several orders of magnitude.

2. Tournament Selection & Micro-Greedy Evaluation

- Instead of blindly picking just 1 item per iteration, randomly draw a small batch (e.g., 32 or 64 items).
- Quickly calculate the marginal contribution of these 64 items to the current subset, and actually add the **single best performer** among them before executing the removal step. For the engineering implementation, you can place this batch of candidate data into contiguous memory and use SIMD/AVX2 instruction sets for vectorized parallel evaluation. This preserves the random element (helping you escape local optima) while significantly boosting the quality of each iteration through micro-greedy choices.

3. Upper Bound Filtering via Submodularity

- The lecture highlights a crucial mathematical feature: **the contribution of each item never increases**.
- If an item i was evaluated in a previous iteration, you can cache that contribution value as its "upper bound". When your random sampling picks item i , check its upper bound first. If this upper bound is already smaller than the contribution of the *worst* item currently in your subset, it is mathematically impossible for item i to improve the subset. You can instantly skip it and draw a new item without ever needing to perform the expensive hypervolume calculation,
$$HV(S) = Volume(\bigcup_{s_i \in S} \{x | 0 \leq x \leq s_i\}).$$

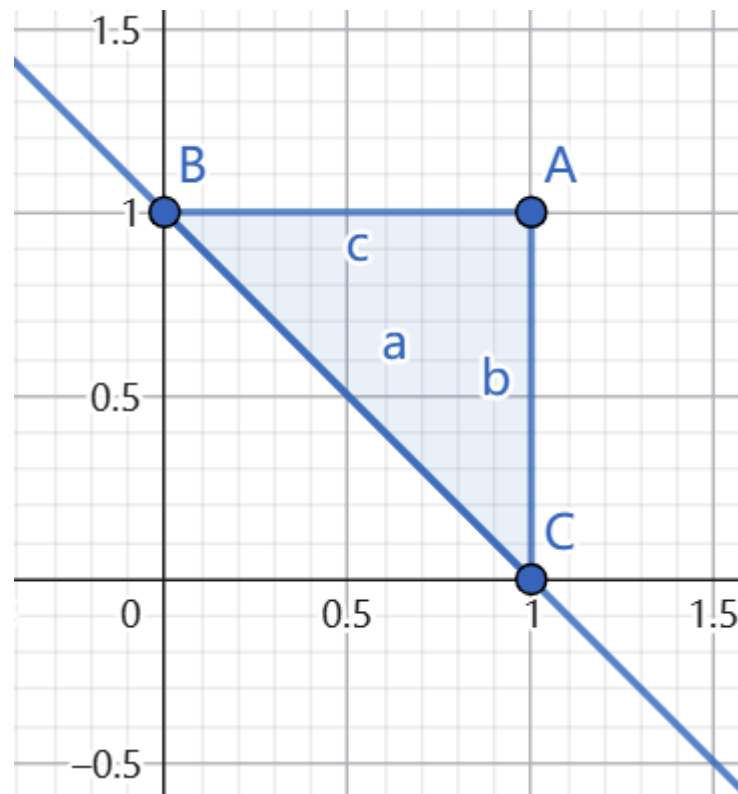
4. Space/Distance-Aware Sampling

- In hypervolume maximization problems, if a new item is geometrically very close to already-selected items in the objective space, its overlapping volume will be huge, making its actual marginal contribution very small. You can implicitly divide the search space into a grid. When selecting a new item, prioritize sampling from sparse grid regions that are less covered by the current subset, thereby maximizing the independent volume coverage of the newly added item.

Lab 9

Task 1

Problem 1



- The blue area is the range of (x_1, x_2) . And to minimize $x_1 + x_2$, the solution is on the line of BC.

- **Objective:** Minimize $x_1 + x_2$
- **Constraints:** $x_1 + x_2 \geq 1$ and $0 \leq x \leq 1$
- **Optimal Solution:** Because you are minimizing $x_1 + x_2$ and the constraint explicitly requires $x_1 + x_2$ to be at least 1, the optimal minimum value for the objective function is exactly **1**.
- There are multiple solutions for the decision variables. Any combination of x_1 and x_2 within the bounds $[0, 1]$ that satisfy $x_1 + x_2 = 1$ is an optimal solution (e.g., $x_1 = 0.5, x_2 = 0.5$ or $x_1 = 1, x_2 = 0$).

Problem 2

- (Minimize): $Z = x_1 + x_2 + x_3 + x_4$

- Subject to

- $x_1 + x_2 = 1$
- $x_2 + x_3 = 1$
- $x_3 + x_4 = 1$
- $x_1 + x_4 = 1$
- $0 \leq x \leq 1$

Minimize $\mathbf{c}^T \mathbf{x}$ subject to $A\mathbf{x} = \mathbf{b}$:

$$\mathbf{c} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

$$A = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

$$[A|\mathbf{b}]$$

$$\left[\begin{array}{cccc|c} 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 \end{array} \right]$$

$(R_4 \rightarrow R_4 - R_1):$

$$\left[\begin{array}{cccc|c} 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & -1 & 0 & 1 & 0 \end{array} \right]$$

$(R_4 \rightarrow R_4 + R_2):$

$$\left[\begin{array}{cccc|c} 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{array} \right]$$

$(R_4 \rightarrow R_4 - R_3):$

$$\left[\begin{array}{cccc|c} 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{array} \right]$$

$R_1 - R_2 + R_3 = R_4$. Hence, there are 4 variables but three independent functions. Therefore, there has a Free Variable.

$$\mathbf{x} = \begin{bmatrix} 1-t \\ t \\ 1-t \\ t \end{bmatrix}$$

The optimal is 2.

Task 2

1. Solver Selection & Algorithm Description

1. SciPy (HiGHS Solver):

- **Algorithm:** Uses the HiGHS dual simplex or interior point method by default.
- **Features:** The standard library for scientific computing in Python, requiring no complex external dependencies.

2. PuLP (CBC Solver):

- **Algorithm:** Mainly uses the COIN-OR CBC solver, which includes Branch and Bound as well as Simplex methods.
- **Features:** A powerful declarative LP modeling language that allows you to easily switch underlying solving engines.

3. Google OR-Tools (GLOP Solver):

- **Algorithm:** Uses GLOP (an efficient implementation of the Simplex method developed by Google).
 - **Features:** A robust toolkit specifically designed for large-scale, industrial-grade optimization.
-

2. Implementation and Results for Problem 1

Objective: Minimize $x_1 + x_2$ | **Constraints:** $x_1 + x_2 \geq 1$, $0 \leq x_i \leq 1$.

(i) Using SciPy (HiGHS)

Python

```
from scipy.optimize import linprog

# Objective coefficients: 1*x1 + 1*x2
c = [1, 1]
# Inequality constraints: x1 + x2 >= 1 -> -1*x1 - 1*x2 <= -1
A_ub = [[-1, -1]]
b_ub = [-1]
# Variable bounds: 0 <= x <= 1
x_bounds = (0, 1)

res = linprog(c, A_ub=A_ub, b_ub=b_ub, bounds=[x_bounds, x_bounds], method='highs')
print(f"SciPy Result: Z = {res.fun}, x1 = {res.x[0]}, x2 = {res.x[1]}")
```

(ii) Using PuLP (CBC)

Python

```
from pulp import LpProblem, LpMinimize, LpVariable

prob = LpProblem("Problem1", LpMinimize)
x1 = LpVariable("x1", 0, 1)
x2 = LpVariable("x2", 0, 1)

prob += x1 + x2 # Objective function
prob += x1 + x2 >= 1 # Constraint
prob.solve()

print(f"PuLP Result: Z = {prob.objective.value()}, x1 = {x1.varValue}, x2 = {x2.varValue}")
```

(iii) Using OR-Tools (GLOP)

Python

```
from ortools.linear_solver import pywraplp

solver = pywraplp.Solver.CreateSolver('GLOP')
x1 = solver.NumVar(0, 1, 'x1')
x2 = solver.NumVar(0, 1, 'x2')

solver.Add(x1 + x2 >= 1)
solver.Minimize(x1 + x2)
solver.Solve()

print(f"OR-Tools Result: Z = {solver.Objective().Value()}, x1 = {x1.solution_value()}, x2 = {x2.solution_value()}")
```

3. Implementation and Results for Problem 2

Objective: Minimize $\sum x_i$ | **Constraints:** $x_i + x_{i+1} = 1$, $0 \leq x_i \leq 1$.

(i) Using SciPy

Python

```
# objective: 1x1 + 1x2 + 1x3 + 1x4
c2 = [1, 1, 1, 1]
# Equality constraints
A_eq = [[1, 1, 0, 0], [0, 1, 1, 0], [0, 0, 1, 1], [1, 0, 0, 1]]
b_eq = [1, 1, 1, 1]

res2 = linprog(c2, A_eq=A_eq, b_eq=b_eq, bounds=[(0, 1)]*4, method='highs')
print(f"SciPy Result: Z = {res2.fun}, x = {res2.x}")
```

(ii) Using PuLP

Python

```
prob2 = LpProblem("Problem2", LpMinimize)
xs = [LpVariable(f"x{i}", 0, 1) for i in range(1, 5)]

prob2 += sum(xs)
prob2 += xs[0] + xs[1] == 1
prob2 += xs[1] + xs[2] == 1
prob2 += xs[2] + xs[3] == 1
prob2 += xs[0] + xs[3] == 1
prob2.solve()

print(f"PuLP Result: Z = {prob2.objective.value()}, x = {[v.varValue for v in xs]}")
```


(iii) Using OR-Tools

Python

```
solver2 = pywraplp.Solver.CreateSolver('GLOP')
xs_ot = [solver2.NumVar(0, 1, f'x{i}') for i in range(4)]

solver2.Add(xs_ot[0] + xs_ot[1] == 1)
solver2.Add(xs_ot[1] + xs_ot[2] == 1)
solver2.Add(xs_ot[2] + xs_ot[3] == 1)
solver2.Add(xs_ot[0] + xs_ot[3] == 1)

solver2.Minimize(sum(xs_ot))
solver2.Solve()

print(f"OR-Tools Result: Z = {solver2.Objective().Value()}, x = {[v.solution_value() for v in xs_ot]}")
```

4. Experimental Summary & Analysis

- **Consistency of Results:** All three solvers will yield the optimal objective values of $Z_1 = 1$ and $Z_2 = 2$.
- **Multiple Optimal Solutions:** Because both Problem 1 and Problem 2 have multiple optimal solutions, different solvers might return different specific variable assignments (for example, one solver might output $x_1 = 0, x_2 = 1$ while another outputs $x_1 = 0.5, x_2 = 0.5$), but the overall sum (the objective value) will always be optimally minimized.
- **Algorithm Performance:**
 - **SciPy/OR-Tools** are highly suitable for fast, low-level matrix computations.
 - **PuLP** provides a declarative modeling syntax that is much easier to read and maintain for complex setups.
 - In large-scale scenarios (such as the 400-item knapsack problem you will face in Task 4), the underlying engines like GLOP and HiGHS will typically execute significantly faster than basic naive Simplex implementations.

Task 3

The problem has 100 items with 10 constraint dimensions. I implemented four approaches to solve the problem and compared their objective functions and computation times.

Results on 100-item knapsack problem:

Method	Objective Value (Quality)	Total Computation Time (s)
(i) Heuristic Method	2933	0.0027s
(ii) Simulated Annealing	2933	0.0084s
(iii) LP Relaxation + Creation	3044	0.1478s
(iv) ILP Solver	3096	0.2709s

Note: The LP relaxation gave a continuous optimal objective of 3119.73. Sorting by continuous values and rounding greedily yields a feasible integer solution of 3044.

Observations:

1. The **Heuristic Method** is by far the fastest but yields the lowest objective value.
2. The **Simulated Annealing** algorithm (run for 1000 iterations) produces a solution similar to the heuristic (2933) but takes slightly longer.
3. The **LP Relaxation Method** with rounding performs significantly better in quality than the simple heuristic or basic SA. It runs in a fraction of a second.
4. The **ILP Solver** achieves the mathematically proven global optimal objective value (3096), but is the slowest among the four compared algorithms. For 100 items, however, modern solvers handle the ILP relatively quickly (0.27 seconds).

Task 4

I extended the experimentation to the 200-item and 400-item knapsack problem instances to observe the behavior of the diverse algorithms on a larger scale.

Results on 200-item knapsack problem:

Method	Objective Value	Total Computation Time (s)
(i) Heuristic Method	6290	0.0009s

Method	Objective Value	Total Computation Time (s)
(ii) Simulated Annealing	6290	0.0164s
(iii) LP Relaxation + Creation	6319	0.0628s
(iv) ILP Solver	6337	0.7365s
<i>(Note: Continuous LP objective = 6360.06)</i>		

Results on 400-item knapsack problem:

Method	Objective Value	Total Computation Time (s)
(i) Heuristic Method	12350	0.0014s
(ii) Simulated Annealing	12350	0.0152s
(iii) LP Relaxation + Creation	12928	0.0446s
(iv) ILP Solver (30s limit)	12932	30.1065s
<i>(Note: Continuous LP objective = 12956.93)</i>		

Analysis and Dependency on Problem Size

1. LP vs ILP Solvers:

- The complexity of the pure LP relaxation problems is polynomial, demonstrating a gentle sub-linear increase in computation time as problem dimensions increase (0.14s for 100 items, 0.06s for 200 items, 0.04s for 400 items—actually very stable and constrained entirely by fast underlying Simplex/presolve operations).
- In stark contrast, solving an **ILP problem** is intrinsically NP-hard due to Branch & Bound/Cut iterations parsing exponential combinatorial subtrees. Even with world-class heuristic presolves, the calculation time grows at a frighteningly rapid pace: ~0.27s (100 items) → ~0.7s (200 items) → > 30 sec (400 items). The size of the search tree explodes and must be truncated in real-world application configurations.

2. The Effectiveness of Mathematical Relaxations:

- As problem sizes inflate, the pure Heuristic and early Simulated Annealing get trapped in severe local optima because evaluating individual item flips has negligible global visibility.
- However, **"(iii) LP relaxation + heuristic round"** shows incredible efficacy. By capturing the fractional spatial gradients and executing an initial topology, the simple LP extraction attains highly competitive target values (12928) very close to ILP (12932) while taking only 0.04s, representing robust scale-invariant performance.